Offline Document Summarizer Project Guide

Version covered: 0.2.0

This document explains how the Offline Document Summarizer works, how the parts connect, and where to make future feature changes. It is written for maintainers who need to understand the project quickly before modifying it.

1. Project Purpose

Offline Document Summarizer is a private Windows desktop app for extracting text from local documents and summarizing that text with a local language model.

The key product promise is:

- Users can drop a document into the app.
- The app extracts text locally.
- Users can review and edit the extracted text.
- The app sends the text only to a local runtime.
- A local Gemma model streams a summary back into the UI.
- Users can save the summary as a local .txt file.

The app does not upload files, extracted text, summaries, or metadata to remote services.

2. Current Technology Stack

The current stack is split into a desktop shell, a web UI, and a local backend.

```
| Layer | Technology | Purpose |
| --- | --- | --- |
| Desktop shell | Tauri 2 + Rust | Creates the Windows desktop app and starts the backend sidecar in
release builds. |
| Frontend | React + TypeScript + Vite | Provides drag-and-drop UI, editable text, settings, setup
check, streaming summary display, and save button. |
| Backend | Python + FastAPI | Handles local extraction, readiness checks, summarization orchestration,
streaming events, and local save fallback. |
| Image OCR | Tesseract + pytesseract + Pillow | Extracts text from PNG, JPG, JPEG, and WEBP files. |
| PDF extraction | pdfplumber, with pypdf fallback | Extracts selectable PDF text page by page. |
| Local model runtime | Ollama | Runs ‘gemma:2b‘ locally through ‘http://127.0.0.1:11434‘. |
| Future model runtime | llama-cpp-python | Loads local GGUF files when llama.cpp mode is selected. |
```

3. Project Structure

```
desktop-summarizer/
  README.md
  .gitignore
  docs/
    PROJECT_DOCUMENTATION.md
    Offline_Document_Summarizer_Project_Guide.pdf
    build_project_pdf.py
  backend/
    main.py
    requirements.txt
    requirements-build.txt
```

```
    requirements-llama-cpp.txt
    extraction/
      image_ocr.py
      pdf_reader.py
      txt_reader.py
    model/
      runtime.py
      ollama_runtime.py
      llama_cpp_runtime.py
    summarizer/
      chat.py
      chunking.py
      prompts.py
      summarize.py
    utils/
      document_store.py
      file_utils.py
      system_check.py
  frontend/
    package.json
    src/
      App.tsx
      api/backend.ts
      components/
        DocumentLibrary.tsx
        DocumentChat.tsx
      styles.css
      types.ts
    src-tauri/
      tauri.conf.json
      Cargo.toml
      src/lib.rs
      icons/
  models/
  outputs/
```

4. Privacy and Offline Boundary

The privacy design is intentional and should be protected when adding features.

Current local-only rules:

- The React UI calls only http://127.0.0.1:8765.
- The FastAPI backend binds only to 127.0.0.1.
- Ollama calls go only to http://127.0.0.1:11434.
- OllamaRuntime rejects non-local Ollama hostnames.
- Files are uploaded only to the local backend process.
- Temporary uploaded files are deleted after extraction.
- Summaries are saved to local disk.
- Document chat uses the current extracted text and recent in-memory chat only.
- The My Documents library stores extracted text, summaries, and metadata under local outputs/documents/.

Do not add remote API calls for document content unless the user explicitly opts into a new feature and the privacy UI is updated.

5. High-Level Runtime Flow

```
User drops file
  -> React DropZone receives File
  -> frontend/src/api/backend.ts sends multipart POST /extract
  -> backend/main.py saves temp file
  -> backend/extraction/* extracts text locally
  -> React displays editable extracted text
  -> User clicks Summarize
  -> React sends POST /summarize with text, mode, length, settings
  -> backend/summarizer/summarize.py chooses direct or chunked summarization
  -> backend/model/runtime.py loads selected runtime
  -> Ollama or llama.cpp streams tokens
  -> FastAPI streams Server-Sent Events to React
  -> React appends tokens into SummaryOutput
  -> User asks a document question
  -> React sends POST /chat with extracted text, question, recent chat, and settings
  -> backend/summarizer/chat.py selects relevant local document context
  -> Ollama or llama.cpp streams answer tokens
  -> React appends tokens into DocumentChat
  -> User clicks Save Document
  -> React sends POST /documents with filename, file type, extracted text, and summary
  -> backend/utils/document_store.py writes local document files under outputs/documents/
  -> User opens an old document from My Documents
  -> React sends GET /documents/{id} and restores extracted text plus saved summary
  -> User clicks Save Summary
  -> Tauri save dialog writes .txt locally, or backend fallback writes under outputs/
```

6. Frontend Architecture

The frontend lives under frontend/src.

Main App State

File: frontend/src/App.tsx

This file coordinates the whole UI:

- active view: workspace or settings
- current settings
- selected filename and file type
- extraction status
- extracted text
- summary text and status
- selected summary mode and length
- document chat messages, draft question, and status
- saved document list and current saved document id
- local setup readiness check
- loading flags and error state

Important handlers:

- handleFileSelected(file) calls local extraction.
- handleSummarize() starts streaming summarization.
- handleAskDocument() starts streaming document Q&A.
- handleSaveDocument() saves or updates the current document in My Documents.

- handleOpenDocument(documentId) restores extracted text and saved summary from My Documents.
- handleSave() saves summary output.
- refreshSystemCheck() checks local prerequisites.

Change this file when:

- You add a new app-wide workflow.
- You add new state that multiple components need.
- You add a new page/tab.
- You change the order of the main workspace layout.

Backend API Wrapper

File: frontend/src/api/backend.ts

This file contains browser-side calls to the Python backend:

- checkSystem(settings) calls POST /system-check.
- extractFile(file) calls POST /extract.
- summarizeStream(...) calls POST /summarize and parses Server-Sent Events.
- chatStream(...) calls POST /chat and parses Server-Sent Events.
- listDocuments(), getDocument(...), saveDocument(...), and deleteDocument(...) call the local document library endpoints.
- saveSummary(summary) uses Tauri's save dialog when available, with backend fallback.

Change this file when:

- You add a new backend route.
- You change request/response shapes.
- You add new streaming event types.
- You change local backend URL configuration.

Shared Types

File: frontend/src/types.ts

Contains TypeScript interfaces and union types:

- SummaryMode
- SummaryLength
- Runtime
- AppSettings
- ExtractResponse
- ChatMessage
- SavedDocumentSummary
- SavedDocumentDetail
- SystemCheckResponse
- SystemCheckItem

Change this file when:

- You add a new summary mode.
- You add a new setting.
- You add a new backend response shape.
- You add a saved document metadata field.
- You add a new runtime.

Components

Folder: frontend/src/components

```
| Component | File | Responsibility |
| --- | --- | --- |
| DropZone | ‘DropZone.tsx‘ | Drag-and-drop file input and Choose File button. |
| FilePreview | ‘FilePreview.tsx‘ | Shows filename, file type, and extraction status. |
| DocumentLibrary | ‘DocumentLibrary.tsx‘ | Shows My Documents, saves the current document, opens old
documents, and deletes saved documents. |
| ExtractedTextPanel | ‘ExtractedTextPanel.tsx‘ | Shows editable extracted text. |
| SummaryControls | ‘SummaryControls.tsx‘ | Summary style dropdown, length dropdown, Summarize, and Save
 Summary. |
| SummaryOutput | ‘SummaryOutput.tsx‘ | Displays streaming summary output and loading state. |
| DocumentChat | ‘DocumentChat.tsx‘ | Displays a local chat session for asking questions about the
extracted document. |
| SetupCheck | ‘SetupCheck.tsx‘ | Shows readiness status for backend, OCR, Ollama, and model. |
| SettingsPage | ‘SettingsPage.tsx‘ | Runtime, model name, GGUF path, context window, and chunk size
settings. |
| PrivacyNote | ‘PrivacyNote.tsx‘ | Shows the local privacy note. |
```

Styling

File: frontend/src/styles.css

This is the central stylesheet. It controls:

- app layout
- responsive breakpoints
- panels and buttons
- setup check cards
- My Documents list
- summary output display
- document chat transcript and input
- text area sizing
- mobile layout

Change this file when:

- You adjust layout.
- You add visual states.
- You add a component and need styling.
- You improve mobile responsiveness.

7. Backend Architecture

The backend lives under backend.

FastAPI Entrypoint

File: backend/main.py

Important routes:

```
| Route | Method | Purpose |
| --- | --- | --- |
| ‘/health‘ | GET | Confirms local backend is running. |
| ‘/system-check‘ | POST | Checks local prerequisites based on current settings. |
| ‘/extract‘ | POST | Receives one file and extracts text locally. |
| ‘/summarize‘ | POST | Streams summary events using the selected runtime. |
| ‘/chat‘ | POST | Streams document Q&A events using the selected runtime. |
| ‘/documents‘ | GET | Lists locally saved documents. |
| ‘/documents/{document_id}‘ | GET | Opens one locally saved document. |
| ‘/documents‘ | POST | Saves or updates one local document record. |
| ‘/documents/{document_id}‘ | DELETE | Deletes one locally saved document. |
| ‘/save-summary‘ | POST | Saves summary text as ‘.txt‘ locally. |
```

Change this file when:

- You add a backend endpoint.
- You change CORS rules.
- You add request models.
- You change upload or streaming behavior.
- You change backend app metadata/version.

Extraction Modules

Folder: backend/extraction

```
| File | Function | Purpose |
| --- | --- | --- |
| ‘image_ocr.py‘ | ‘extract_text_from_image(file_path)‘ | Uses Pillow and Tesseract OCR for image files.
 |
| ‘pdf_reader.py‘ | ‘extract_text_from_pdf(file_path)‘ | Uses ‘pdfplumber‘, then ‘pypdf‘, to extract
selectable PDF text. |
| ‘txt_reader.py‘ | ‘extract_text_from_txt(file_path)‘ | Reads TXT with UTF-8 and fallback encodings. |
```

Change these files when:

- You add a new input format.
- You improve OCR preprocessing.
- You add scanned-PDF OCR fallback.
- You improve encoding detection.
- You change extraction error messages.

File Utilities

File: backend/utils/file_utils.py

Responsibilities:

- supported extension lists
- file type detection
- AppError application exception
- local summary saving

Change this file when:

- You add supported file extensions.
- You change output filename behavior.
- You want centralized validation logic.

Document Store

File: backend/utils/document_store.py

Responsibilities:

- list saved documents from outputs/documents/
- save extracted text, summary text, and metadata locally
- open one saved document by id
- delete one saved document by id
- prevent path traversal with safe document ids

Saved document layout:

```
outputs/documents/{document_id}/
  metadata.json
  extracted.txt
  summary.txt
```

Change this file when:

- You add tags, folders, search, or favorites.
- You add original-file copying.
- You change saved document metadata.
- You add import/export for the local library.

System Readiness Check

File: backend/utils/system_check.py

This is the first-run setup checker. It checks:

- local backend is running
- Tesseract availability
- Ollama availability
- whether selected Ollama model is installed
- optional llama.cpp dependency and GGUF path

Change this file when:

- You add a new runtime.
- You add a dependency that should be checked before use.
- You want better setup instructions in the UI.
- You change default model requirements.

Model Runtime Selector

File: backend/model/runtime.py

This selects the model runtime from settings:

- ollama -> backend/model/ollama_runtime.py
- llama.cpp -> backend/model/llama_cpp_runtime.py

Change this file when:

- You add a new model backend.
- You rename runtime values.
- You change runtime selection rules.

Ollama Runtime

File: backend/model/ollama_runtime.py

Responsibilities:

- enforces local-only Ollama host
- sends prompt to /api/generate
- streams response tokens
- maps Ollama errors into friendly AppError messages

Important defaults:

- URL: http://127.0.0.1:11434
- model: gemma:2b
- temperature: 0.2
- top_p: 0.9
- context: from settings

Change this file when:

- You change generation parameters.
- You add stop sequences.
- You add model warmup.
- You support a different Ollama endpoint.
- You improve model-not-found handling.

llama.cpp Runtime

File: backend/model/llama_cpp_runtime.py

This is already scaffolded for local GGUF models through llama-cpp-python.

It validates:

- GGUF path is provided
- file exists
- llama_cpp can be imported

Change this file when:

- You fully invest in GGUF support.
- You expose CPU/GPU layer settings.
- You tune n_threads, n_ctx, temperature, or max_tokens.
- You add model caching.

8. Summarization Pipeline

Folder: backend/summarizer

Prompt Building

File: backend/summarizer/prompts.py

Current modes:

- concise
- bullets
- key_ideas
- study_notes
- action_items
- eli10
- meeting_notes
- research_paper
- legal_policy
- email
- x_thread
- reddit_linkedin

Current lengths:

- short
- medium
- detailed

The build_prompt() function adds:

- a length hint
- an instruction to return only the summary

- mode-specific prompt text

Change this file when:

- You add a summary style.
- You tune prompt wording.
- You add a new output format.
- You want different prompts per model/runtime.

Chunking

File: backend/summarizer/chunking.py

The current token estimate is simple:

```
estimated tokens = characters / 4
```

chunk_text() splits large text by paragraphs first, then by sentences, then by fixed slices if needed.

Change this file when:

- You need more accurate token counting.
- You add tokenizer-based chunking.
- You want overlap between chunks.
- You want page-aware chunking.

Summary Orchestration

File: backend/summarizer/summarize.py

Important functions:

- summarize_text(text, mode, length, settings)
- summarize_large_document(text, mode, length, settings)
- iter_summary_events(text, mode, length, settings)

Direct summarization path:

```
Estimate token count
If text fits context:
  build prompt
  stream model tokens
```

Large document path:

```
Estimate token count
If text is too large:
  split into chunks
  summarize each chunk using short length
  combine chunk summaries
  run final summary over combined summaries
  stream final output
```

Change this file when:

- You change large-document strategy.
- You add progress percentages.
- You stream chunk summaries into the UI.
- You add cancellation support.
- You add caching.

Document Chat Orchestration

File: backend/summarizer/chat.py

Important functions:

- iter_chat_events(text, question, history, settings)
- select_relevant_context(text, question, settings, safe_input_tokens)
- build_document_chat_prompt(context, question, history)

Direct chat path:

```
Validate extracted text and user question
If text fits context:
  use the full extracted text as document context
  stream model answer tokens
```

Large document chat path:

```
Estimate token count
If text is too large:
  split text into chunks
  score chunks against question keywords
  select the highest-scoring chunks within the context budget
  stream the model answer using selected excerpts
```

The chat prompt tells the model to answer only from the document context. If the answer is not present, the model should say it cannot find the answer in the document.

Change this file when:

- You improve document Q&A retrieval.
- You add embeddings or semantic search.
- You change how much chat history is included.
- You tune the document chat prompt.
- You add citations or page-aware references.

9. API Contract

GET ‘/health‘

Returns:

```
{
```

```
  "status": "ok",
  "privacy": "local-only",
  "ollamaApi": "http://127.0.0.1:11434"
}
```

POST ‘/system-check‘

Request body:

```
{
  "runtime": "ollama",
  "modelName": "gemma:2b",
  "ggufModelPath": "",
  "contextWindow": 4096,
  "chunkSize": 9000
}
```

Returns:

```
{
  "allReady": true,
  "checkedAt": "2026-05-02T16:12:04",
  "items": [
    {
      "id": "backend",
      "label": "Local backend",
      "status": "ready",
      "message": "Running on 127.0.0.1 only.",
      "detail": ""
    }
  ]
}
```

POST ‘/extract‘

Request:

- multipart form
- field name: file

Returns:

```
{
  "filename": "notes.pdf",
  "fileType": "pdf",
  "status": "Text extracted locally",
  "text": "..."
}
```

POST ‘/summarize‘

Request body:

```
{
  "text": "extracted text",
  "mode": "concise",
  "length": "medium",
  "settings": {
```

```
    "runtime": "ollama",
    "modelName": "gemma:2b",
    "ggufModelPath": "",
    "contextWindow": 4096,
    "chunkSize": 9000
  }
}
```

Response:

- text/event-stream
- event names: status, token, error, done

Example events:

```
event: status
data: {"type":"status","message":"Summarizing locally..."}

event: token
data: {"type":"token","text":"The document explains..."}
```

POST ‘/chat‘

Request body:

```
{
  "text": "extracted text",
  "question": "What are the action items?",
  "history": [
    {
      "role": "user",
      "content": "What is this document about?"
    },
    {
      "role": "assistant",
      "content": "It explains..."
    }
  ],
  "settings": {
    "runtime": "ollama",
    "modelName": "gemma:2b",
    "ggufModelPath": "",
    "contextWindow": 4096,
    "chunkSize": 9000
  }
}
```

Response:

- text/event-stream
- event names: status, token, error, done

Example events:

```
event: status
data: {"type":"status","message":"Finding relevant document context locally..."}

event: token
data: {"type":"token","text":"The document lists three action items..."}
```

GET ‘/documents‘

Returns:

```
{
  "documents": [
    {
      "id": "7a6f...",
      "filename": "Biology chapter.pdf",
      "fileType": "PDF",
      "createdAt": "2026-05-21T13:12:04+00:00",
      "updatedAt": "2026-05-21T13:20:11+00:00",
      "textChars": 18640,
      "summaryChars": 1240
    }
  ]
}
```

GET ‘/documents/{document_id}‘

Returns one saved document, including extracted text and summary:

```
{
  "id": "7a6f...",
  "filename": "Biology chapter.pdf",
  "fileType": "PDF",
  "createdAt": "2026-05-21T13:12:04+00:00",
  "updatedAt": "2026-05-21T13:20:11+00:00",
  "textChars": 18640,
  "summaryChars": 1240,
  "text": "extracted text",
  "summary": "saved summary"
}
```

POST ‘/documents‘

Creates or updates one saved document. If id is omitted or null, a new saved document is created.

Request body:

```
{
  "id": null,
  "filename": "Meeting notes.txt",
  "fileType": "TXT",
  "text": "extracted text",
  "summary": "saved summary"
}
```

Returns the saved document detail.

DELETE ‘/documents/{document_id}‘

Deletes the saved document folder from outputs/documents/.

Returns:

```
{
  "status": "Saved document deleted locally"
}
```

POST ‘/save-summary‘

Request body:

```
{
  "summary": "summary text",
  "outputPath": null
}
```

Returns:

```
{
  "path": "D:\\offline_llm\\desktop-summarizer\\outputs\\summary-20260502-161204.txt",
  "status": "Summary saved locally"
}
```

10. Tauri Packaging

Tauri files live under frontend/src-tauri.

Important files:

```
| File | Purpose |
| --- | --- |
| ‘tauri.conf.json‘ | App metadata, bundle config, CSP, sidecar config, window config. |
| ‘Cargo.toml‘ | Rust package metadata and Tauri plugin dependencies. |
| ‘src/lib.rs‘ | Registers plugins and starts backend sidecar in release builds. |
| ‘capabilities/default.json‘ | Tauri permissions for dialog, filesystem, and shell plugin. |
| ‘icons/icon.ico‘ | Windows app and installer icon. |
```

Release behavior:

- In development, npm run tauri:dev starts the Python backend with dev:backend.
- In release builds, Tauri starts the bundled backend sidecar.
- The sidecar binary must be named with the Windows target suffix:

```
backend-dist/offline-summarizer-backend-x86_64-pc-windows-msvc.exe
```

11. Build and Release Workflow

Development Run

From frontend/:

```
npm run tauri:dev
```

Build Backend Sidecar

From backend/:

```
..\.venv\Scripts\python.exe -m PyInstaller --name offline-summarizer-backend --onefile main.py --clean
```

Then copy:

```
Copy-Item backend\dist\offline-summarizer-backend.exe backend-dist\offline-summarizer-backend-x86_64-pc-
windows-msvc.exe -Force
```

Build Windows App

From frontend/:

```
npm run tauri:build
```

If Git's link.exe is picked instead of MSVC's linker, build from the Visual Studio C++ developer environment or prepend the MSVC linker directory to PATH.

Release Artifacts

Current release artifacts are created under:

```
frontend/src-tauri/target/release/bundle/
```

Recommended public download:

```
frontend/src-tauri/target/release/bundle/nsis/Offline Document Summarizer_0.2.0_x64-setup.exe
```

MSI alternative:

```
frontend/src-tauri/target/release/bundle/msi/Offline Document Summarizer_0.2.0_x64_en-US.msi
```

12. Where To Make Common Feature Changes

Add a New File Type

Files to change:

- backend/utils/file_utils.py
- add a new extraction module under backend/extraction/
- backend/main.py
- frontend/src/components/DropZone.tsx
- possibly frontend/src/App.tsx

Example:

To add DOCX support:

1. Add .docx to supported extensions in file_utils.py.
2. Create backend/extraction/docx_reader.py.
3. Route file_type == "docx" in backend/main.py.
4. Add .docx to the file input accept list in DropZone.tsx.

Improve OCR Quality

Files to change:

- backend/extraction/image_ocr.py

Possible improvements:

- deskew images
- increase contrast
- threshold image to black/white
- add language selection
- expose OCR language in settings

Add Scanned PDF OCR

Files to change:

- backend/extraction/pdf_reader.py
- possibly add pdf2image or PyMuPDF
- backend/requirements.txt
- README.md
- backend/utils/system_check.py

Suggested flow:

1. Try selectable text extraction.
2. If no text is found, render PDF pages as images.
3. Run local OCR per page.
4. Return page-labeled OCR text.

Add a New Summary Style

Files to change:

- backend/summarizer/prompts.py
- frontend/src/types.ts
- frontend/src/components/SummaryControls.tsx

Example:

1. Add a new key to MODE_PROMPTS.
2. Add the new union value to SummaryMode.
3. Add a new dropdown option in SummaryControls.tsx.

Add a New Length Option

Files to change:

- backend/summarizer/prompts.py

- frontend/src/types.ts
- frontend/src/components/SummaryControls.tsx

Add a New Runtime

Files to change:

- backend/model/runtime.py
- create a new runtime file under backend/model/
- backend/utils/system_check.py
- frontend/src/types.ts
- frontend/src/components/SettingsPage.tsx

Change Ollama Model Defaults

Files to change:

- frontend/src/App.tsx for DEFAULT_SETTINGS
- backend/main.py for backend default Settings
- README.md for setup instructions
- backend/utils/system_check.py if readiness copy should change

Change Prompt Behavior

Files to change:

- backend/summarizer/prompts.py
- backend/model/ollama_runtime.py for system instruction

Change Chunking Behavior

Files to change:

- backend/summarizer/chunking.py
- backend/summarizer/summarize.py
- frontend/src/components/SettingsPage.tsx if settings need to change

Add Progress Percentage

Files to change:

- backend/summarizer/summarize.py
- backend/main.py only if event shape changes
- frontend/src/api/backend.ts
- frontend/src/components/SummaryOutput.tsx

Add Cancel Summarization

Files to change:

- frontend/src/api/backend.ts
- frontend/src/App.tsx
- frontend/src/components/SummaryControls.tsx
- backend may need request/session IDs if cancellation should stop the model call, not only ignore output

Improve Settings Persistence

Current settings are stored in browser localStorage.

Files to change:

- frontend/src/App.tsx
- possibly add a Tauri filesystem-backed settings file
- frontend/src/api/backend.ts if backend persistence is added

Change Save Behavior

Files to change:

- frontend/src/api/backend.ts
- backend/utils/file_utils.py
- backend/main.py

Current behavior:

- In Tauri, save uses a native save dialog.
- Outside Tauri, backend fallback writes to outputs/.

Change My Documents Behavior

Files to change:

- backend/utils/document_store.py
- backend/main.py
- frontend/src/api/backend.ts
- frontend/src/components/DocumentLibrary.tsx
- frontend/src/App.tsx
- frontend/src/types.ts

Current behavior:

- Saved documents are stored locally under outputs/documents/.
- Each saved document has metadata, extracted text, and summary text.
- Opening a saved document restores extracted text and saved summary.

Change UI Layout

Files to change:

- frontend/src/App.tsx
- frontend/src/styles.css
- component files under frontend/src/components/

Add App Auto-Update

Files to change:

- Tauri updater configuration
- frontend/src-tauri/tauri.conf.json
- release signing keys and GitHub release process

This feature has security implications and should be planned carefully.

13. Error Handling Map

```
| Error | Where It Comes From | User-Facing Behavior |
| --- | --- | --- |
| Unsupported file type | ‘backend/main.py‘, ‘file_utils.py‘ | Upload fails with supported extensions
message. |
| OCR failed | ‘backend/extraction/image_ocr.py‘ | User sees OCR error. |
| Tesseract missing | ‘image_ocr.py‘, ‘system_check.py‘ | Setup check marks OCR missing; extraction
shows OCR failure. |
| PDF extraction failed | ‘backend/extraction/pdf_reader.py‘ | User sees PDF extraction error. |
| Empty extracted text | ‘backend/main.py‘, ‘summarize.py‘ | User sees empty text message. |
| Ollama not running | ‘ollama_runtime.py‘, ‘system_check.py‘ | Setup check marks Ollama missing;
summarization fails gracefully. |
| Model missing | ‘ollama_runtime.py‘, ‘system_check.py‘ | Setup check shows ‘ollama pull model‘;
summarization error includes pull command. |
| Context too small | ‘summarize.py‘, ‘llama_cpp_runtime.py‘ | User is told to increase context window
or reduce chunk size. |
| Save failed | ‘file_utils.py‘, ‘backend.ts‘ | User sees local save error. |
| Saved document missing | ‘document_store.py‘ | User sees that the saved document was not found. |
```

14. Important Defaults

```
| Setting | Default | Where Defined |
| --- | --- | --- |
| Runtime | ‘ollama‘ | ‘frontend/src/App.tsx‘, ‘backend/main.py‘ |
| Model | ‘gemma:2b‘ | ‘frontend/src/App.tsx‘, ‘backend/main.py‘ |
| Context window | ‘4096‘ | ‘frontend/src/App.tsx‘, ‘backend/main.py‘ |
| Chunk size | ‘9000‘ characters | ‘frontend/src/App.tsx‘, ‘backend/main.py‘ |
| Backend URL | ‘http://127.0.0.1:8765‘ | ‘frontend/src/api/backend.ts‘ |
| Ollama URL | ‘http://127.0.0.1:11434‘ | ‘backend/model/ollama_runtime.py‘,
‘backend/utils/system_check.py‘ |
| Reserved output tokens | ‘900‘ | ‘backend/summarizer/summarize.py‘ |
| Saved documents folder | ‘outputs/documents/‘ | ‘backend/main.py‘, ‘backend/utils/document_store.py‘ |
```

15. Testing Checklist Before Release

Run these checks before publishing a new version:

```
.\.venv\Scripts\python.exe -m compileall backend
cd frontend
npm run build
```

```
npm audit --omit=dev
```

Backend manual checks:

- Start backend.
- Open http://127.0.0.1:8765/health.
- Call /system-check.
- Extract from TXT.
- Extract from PDF.
- Extract from image OCR.
- Summarize a short document.
- Summarize a large document.
- Save summary.

Release build checks:

- Rebuild PyInstaller sidecar.
- Copy sidecar to backend-dist/.
- Run npm run tauri:build.
- Launch release executable.
- Confirm setup check is visible.
- Confirm Summarize streams output.
- Confirm installer files are under frontend/src-tauri/target/release/bundle/.

16. Current Limitations

- OCR only handles image files, not scanned PDFs directly.
- Token estimation is approximate.
- Ollama must be installed separately.
- gemma:2b must be pulled separately.
- llama.cpp mode is scaffolded but not as polished as Ollama mode.
- There is no cancellation button for long generations.
- There is no document preview image/PDF viewer, only filename/type/status and extracted text.
- There is no auto-update system.

17. Suggested Roadmap

High-impact next features:

1. Scanned PDF OCR fallback.
2. First-run installer/helper page with direct setup links.
3. Better model selector using installed Ollama models.
4. Cancel summarization button.
5. Progress percentage for chunked summarization.
6. Summary history stored locally.
7. Export as Markdown and PDF.
8. App auto-update after code signing is planned.

18. Maintenance Notes

Keep the project maintainable by following these rules:

- Preserve the local-only privacy boundary.
- Keep extraction logic in backend/extraction.
- Keep prompt logic in backend/summarizer/prompts.py.
- Keep runtime-specific model logic in backend/model.
- Keep frontend API calls centralized in frontend/src/api/backend.ts.
- Keep shared frontend types in frontend/src/types.ts.
- Update this document when you add a new runtime, file type, API endpoint, or release workflow.